MyCodeSchool

Spring Data JPA — fișă rapidă

One Model · aceeași numerotare ca în fișa de exerciții: la exercițiul N citești caseta N

Înainte de exerciții · cum lucrezi și unde scrii

```
docker compose up -d      pornește MySQL
mvn spring-boot:run       sau Run din IntelliJ
docker compose down -v    șterge baza, o iei de la zero

în consolă:  Hibernate: select ... where ...
             ──► SQL-ul scris în locul tău. Citește-l.
```

```
un exercițiu ──► o metodă în ViewProduct
             ──► o chemi în start() ──► rulezi
             ──► „Gata când” din fișă?
                   da ──► commit ──► următorul
                   nu ──► citești SQL-ul + „Caused by:”
```

```
ViewProduct ──────────► ProductRepository ──────────► Hibernate ──────► MySQL
 start() + o metodă      interfață: scrii DOAR        scrie SQL-ul      tabela products
 per exercițiu           semnătura metodei                              un Product = un rând
 AICI chemi și afișezi   AICI adaugi metode (ex. 4–10)                  addedOn = coloana added_on

Product + Category = modelul: îl primești gata, NU îl modifici.      În casete, repo = productRepository.
```

1 · Populare · deleteAll · saveAll · count

```
repo.deleteAll();                 golește tabela
repo.saveAll(List.of(p1, p2));    scrie toate, un apel
repo.count();                     ──► long

new Product("Espresso", "COF-0001", Category.COFFEE,
    new BigDecimal("9.50"), 120, true,
    LocalDate.of(2026, 1, 10));
```

```
save / saveAll, pentru fiecare produs:
  │ are id ?
  ├─ nu ──► insert  (primește id nou)
  └─ da ──► update  (vezi 8)

✗ fără deleteAll, a doua rulare pică:
  Duplicate entry 'COF-0001' (sku e unic)
```

2 · Citire · findAll · findById · Optional · existsById

```
List<Product> toate = repo.findAll();
UUID id = toate.get(0).getId();

Optional<Product> gasit = repo.findById(id);
boolean este = repo.existsById(id);
```

```
findById(id) ──► Optional<Product>
                   ├─ isPresent() ──► get() ──► Product
                   └─ isEmpty()   ──► nu există
✗ get() pe un Optional gol aruncă excepție
✗ findAll nu sortează: ordinea poate varia
```

3 · Validări și sku unic · saveAndFlush · ConstraintViolationException · DataIntegrityViolationException

```
try {
    repo.saveAndFlush(gresit);
} catch (ConstraintViolationException e) {
    e.getConstraintViolations().stream()
        .map(violation -> violation.getMessage())
        .sorted()
        .forEach(System.out::println);
}
```

```
saveAndFlush(p)
  │ 1. regulile din Product (@NotBlank, @Min...)
  │       ✗ ──► ConstraintViolationException
  │             jakarta.validation · fără insert
  │ 2. insert ──► MySQL verifică unique pe sku
  │       ✗ ──► DataIntegrityViolationException
  ▼ salvat      org.springframework.dao
✗ cu save, pasul 1 pică abia la final și oprește tot
```

4 · Metode derivate, ghidat · findBy · existsBy · countBy

```
în ProductRepository scrii doar semnătura:

Optional<Product> findBySku(String sku);
   find    By    Sku
    │            │
    ce face      câmpul din Product
    ▼
select ... from products where sku = ?
```

```
find...By   ──► Optional (cel mult unul) sau List
exists...By ──► boolean
count...By  ──► long

enum ca parametru:  findByCategory(Category.PASTRY)

✗ câmp scris greșit ──► aplicația nu pornește:
  No property 'skuu' found for type 'Product'
```

5 · Metode derivate, singur · cuvintele-cheie

```
regula vine după câmp:          în SQL devine:

findByStockBetween(a, b)        stock between ? and ?
findByPriceLessThan(x)          price < ?
findByPriceGreaterThanEqual(x)  price >= ?
findByAddedOnBefore(data)       added_on < ?
findBySkuContaining(text)       sku like %text%
findByStockIn(lista)            stock in (?, ?, ?)
findByActiveFalse()             active = false
```

```
două condiții:
findByNameAndStock(name, stock)
          │    └── al doilea parametru
          └─────── primul parametru

fără literă mare / mică:  ...ContainingIgnoreCase(text)

cum gândești: 1. ce câmp?  2. ce regulă?
              3. mai e o condiție? ──► And
              4. câte valori cere regula = nr. parametri
```

6 · Sortare, limitare, paginare · OrderBy · Top · Sort · PageRequest · Page

```
în nume:
findByActiveTrueOrderByNameAsc()    sortat
findTop5ByOrderByStockAsc()         primele 5
findFirstByOrderByStockAsc()        primul

ca parametru, la metodele pe care le ai deja:
repo.findAll(Sort.by("name"));
repo.findAll(Sort.by(Sort.Direction.DESC, "stock"));
```

```
10 produse, câte 4:  [0: 1–4] [1: 5–8] [2: 9–10]

PageRequest.of(1, 4, Sort.by("name"))
               │  └ câte pe pagină
               └ numărul paginii, de la 0
Page<Product> p = repo.findAll(pageRequest);
p.getContent() · getTotalElements() · getTotalPages()

metodă proprie cu pagini:  Page<Product>
    findBy...(Pageable pageable);
```

7 · @Query · @Param · JPQL · nativeQuery

```
JPQL = vorbești cu clasa

@Query("select p from Product p "
     + "where p.price <= :maxPrice order by p.price")
List<Product> findUpTo(
        @Param("maxPrice") BigDecimal maxPrice);

:maxPrice ◄──── @Param("maxPrice")   același nume
calcule:  select sum(p.stock) ...   avg(p.price)
```

```
SQL nativ = vorbești cu tabela

@Query(value = "SELECT * FROM products "
             + "WHERE sku LIKE CONCAT(:text, '%')",
       nativeQuery = true)

              JPQL            SQL nativ
tabelă        Product p       products
coloană       p.addedOn       added_on
```

8 · Modificare · save ca update · @Modifying · @Transactional

```
un produs:
Product p = repo.findBySku("COF-0001").get();
p.setPrice(new BigDecimal("10.00"));
repo.save(p);        are id ⟶ update, nu insert
```

```
mai multe rânduri, dintr-un singur SQL:
@Modifying(clearAutomatically = true)
@Transactional
@Query("update Product p set p.x = ... where ...")
int metoda();        ⟶ câte rânduri a schimbat
✗ @Transactional = org.springframework.transaction
                   .annotation, nu jakarta
```

9 · Ștergere · deleteById · deleteBy...

```
repo.deleteById(p.getId());     un rând, după id

@Transactional
long deleteByActiveFalse();     după o condiție
     ⟶ câte a șters
```

```
deleteBy...() ⟶ select (le încarcă)
              ⟶ câte un delete pe rând
✗ fără @Transactional:
  No EntityManager with actual transaction
```

10 · Provocări · nimic nou: alegi caseta

```
„cel mai ...”  ⟶ sortezi și iei primul: First + OrderBy (6)
„într-o lună”  ⟶ Between două date (5)
„câte ...”          ⟶ countBy + condiții (4, 5)
„media / suma ...”  ⟶ @Query cu avg / sum (7)
```